

INTERVIEW GUIDE • BOOK 01

JAVASCRIPT

BASICS → INTERMEDIATE

A structured learning path for the frontend interview. Read in order — each part builds on the one before it. Every topic carries a definition, working code, and the question interviewers actually ask.

05

PARTS

26

TOPICS

12

PAGES

PAIRS WITH BOOK 02 — REACT & FULL-STACK

2026 EDITION

CONTENTS

Learn in this exact order. Parts 1–2 are the ground you stand on; Parts 3–5 are where interviews are won.

01	WEB FOUNDATIONS	Page 03
Folders & files · browsers · URL · HTTP/HTTPS · SSL · DNS · text editors & agents		
02	HTML & CSS	Page 04
Tags & elements · semantic vs non-semantic · attributes · forms · 3 CSS types · selectors · box model · flex · grid		
03	JAVASCRIPT CORE	Page 06
Variables · datatypes · operators · implicit/explicit coercion · conditions & control · statements vs expressions · objects & arrays		
04	FUNCTIONS, ARRAYS & CLOSURES	Page 08
Parameters vs arguments · arrow functions · callbacks · prototypes & built-ins · array methods & HOF · closures		
05	ASYNC & BROWSER PATTERNS	Page 10
JSON · promises & chaining · fetch & axios · try/catch · async/await · bubbling & delegation · debounce & throttle · scale concepts		

HOW TO USE THIS BOOK

One part per sitting. Type every code block yourself — don't copy-paste. After each part, answer the Interview Q boxes out loud, as if the interviewer is in the room. Revise the boxes alone the night before the interview.

HOW THE WEB WORKS

FOLDER & FILE SYSTEM

Every project is a folder tree: files hold code, folders organize it. Your text editor (VS Code) opens the folder as a workspace; AI agents and the terminal operate on the same tree. Paths connect files: `./` same folder, `../` one level up.

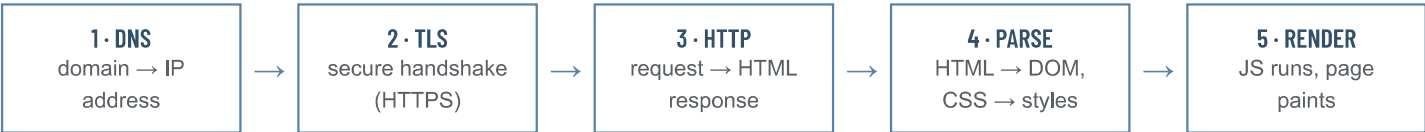
my-project/
├ index.html
├ styles/style.css
└ scripts/app.js

← entry page the browser opens

← linked: <link rel="stylesheet" href="styles/style.css">

← linked: <script src="scripts/app.js"></script>

FROM URL TO PIXELS – THE FLOW TO NARRATE IN INTERVIEWS



THE VOCABULARY – ONE LINE EACH

URL	Address of a resource. Anatomy: <code>https://www.shop.com/products?sort=price</code> = protocol + subdomain + domain + path + query.
HTTP	The request/response protocol between client and server. Stateless — each request stands alone.
HTTPS	HTTP encrypted with TLS/SSL — data cannot be read or altered in transit. The padlock in the browser.
SSL / TLS	The certificate-based encryption layer under HTTPS. The server proves its identity with a certificate.
DNS	Phonebook of the internet: resolves <code>google.com</code> → an IP like <code>142.250.1.1</code> before any request can be sent.
WWW	A conventional subdomain; <code>www.site.com</code> and <code>site.com</code> usually serve the same app.
Browser	Parses HTML into the DOM and runs JS on an engine: V8 (Chrome/Edge), SpiderMonkey (Firefox), JavaScriptCore (Safari).

INTERVIEW Q – THE CLASSIC OPENER

"What happens when you type a URL and press Enter?" — Narrate the 5-step flow above, in order, using the words DNS, TLS, HTTP, DOM, render. Sixty seconds, no more.

HTML – STRUCTURE OF THE PAGE

TAGS, ELEMENTS, ATTRIBUTES

A **tag** is the markup itself (<p>). An **element** is opening tag + content + closing tag. Void elements (<input />, ,
) carry no children. An **attribute** configures the element: attributename="value".

```
<html>                ← ROOT element
  <head> ...meta, title, links... </head>
  <body>                ← everything visible
    <input type="email" placeholder="you@mail.com" required />
    <a href="https://site.com" target="_blank">Visit</a>
  </body>
</html>
```

Container elements hold other elements — div, section, form, ul, table. A "card" is just a div composing image + title + text + button.

THE FOUR DISTINCTIONS INTERVIEWERS TEST

CATEGORY	EXAMPLES	WHY IT MATTERS
Semantic	header nav main section article aside footer	Meaning → SEO, screen readers
Non-semantic	div span	Generic boxes, no meaning
Block-level	div p h1-h6 ul ol table form	Full width, starts a new line
Inline	span a img strong em input	Flows in text; width/height ignored unless inline-block

FORMS, LISTS & TABLES – KNOW COLD

```
<form>                <ul> <li>bullet item</li> </ul>
  <label for="em">Email</label>    <ol> <li>numbered item</li> </ol>
  <input id="em" name="email" />
  <button type="submit">Send</button>  <table>
</form>                <thead><tr><th>Head</th></tr></thead>
                                <tbody><tr><td>Cell</td></tr></tbody>
                                </table>
```

INTERVIEW Q

"Why semantic tags over divs?" — Accessibility (screen readers navigate by landmarks), SEO (search engines weigh <main>/<article>), and readable code. A page of divs works, but tells machines nothing.

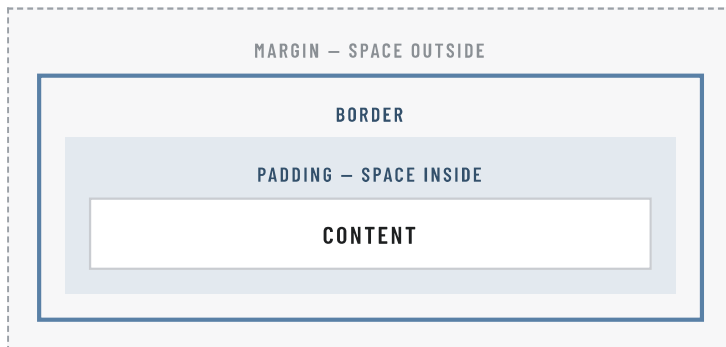
CSS — SELECTORS, BOX MODEL, LAYOUT

3 WAYS TO APPLY · SELECTOR SYNTAX

Inline (style="", highest priority) → **Internal** (<style> in head) → **External** (<link>, the professional standard). Specificity order: inline > #id > .class > element.

```
selector { propertyname: value; }
.card { } #hero { } button { } input[type="email"] { } .btn:hover { }
.card h3 { } /* descendant */ .card > h3 { } /* direct child */
```

THE BOX MODEL — CONTENT → PADDING → BORDER → MARGIN



Every element is a box. By default width means the content only — padding and border add on top. Set box-sizing: border-box globally so width means the whole visible box. Interviewers ask exactly this: *"width: 200px + padding: 20px — how wide is the element?"* Default: 240px. With border-box: 200px.

FLEXBOX (1-D) VS GRID (2-D)

FLEX — ONE ROW OR COLUMN



```
display: flex;
justify-content: center; /* main axis */
align-items: center; /* cross axis */
gap: 16px; flex-wrap: wrap;
```

GRID — ROWS AND COLUMNS



```
display: grid;
grid-template-columns: repeat(3, 1fr);
gap: 16px;
grid-column: span 2; /* on a child */
```

INTERVIEW Q

"Center a div?" — Give two answers: flex (justify-content:center; align-items:center) and grid (place-items:center). "Flex or grid?" — flex for a line of items, grid for a 2-D layout of cells.

VARIABLES, TYPES & COERCION

VAR · LET · CONST

	VAR (LEGACY)	LET	CONST (DEFAULT)
Scope	function	block { }	block { }
Re-assign	yes	yes	no (object contents can still change)
Hoisting	hoisted as undefined	hoisted into the Temporal Dead Zone — ReferenceError if used early	

DATATYPES – VALUE VS REFERENCE

Primitive (7): string · number · boolean · undefined · null · symbol · bigint — copied *by value*. **Non-primitive:** object · array · function — copied *by reference*. This one distinction explains half of React’s rules later.

```
let x = 5; let y = x; y = 9;           // x is still 5      (copied value)
let a = { n: 1 }; let b = a; b.n = 9; // a.n is 9!         (same reference)
typeof null           // "object" ← historic bug, guaranteed question
typeof undefined      // "undefined" – null is intentional empty, undefined is "not set yet"
```

OPERATORS & COERCION – IMPLICIT VS EXPLICIT

```
// IMPLICIT – JS converts behind your back
"5" + 1    // "51"    + prefers string concatenation
"5" - 1    // 4       - forces numbers
1 == "1"   // true    loose equality coerces first
1 === "1"  // false   strict equality: value AND type – always use ===

// EXPLICIT – you convert on purpose
Number("5") String(5) Boolean(0) parseInt("42px") // 42
```

The six **falsey** values — memorize: false · 0 · "" · null · undefined · NaN. Everything else is truthy, including "0", [] and {}.

INTERVIEW Q

"var vs let vs const?" — answer with the three rows of the table: scope, re-assignment, hoisting. Then add: "I default to const, use let when re-assignment is needed, and never var."

CONTROL FLOW, OBJECTS & ARRAYS

CONDITIONS & LOOPS

```
if (score >= 90) grade = "A";
else if (score >= 75) grade = "B";
else grade = "C";

switch (day) { case "Mon": start(); break; default: rest(); }

for (let i = 0; i < 5; i++) { }      while (n > 0) { n--; }
for (const item of arr) { }         // of → VALUES of an array
for (const key in obj) { }          // in → KEYS of an object
```

STATEMENTS VS EXPRESSIONS

A **statement** performs an action (if, for, declarations) and produces no value. An **expression** produces a value: `2 + 2`, `a ? b : c`, `fn()`. The distinction matters in React: JSX braces accept *only expressions* — which is why you use ternaries there, never if.

OBJECTS — DECLARE & ACCESS

```
const user = {
  name: "Asha",
  "job title": "developer",      // key with a space
  address: { city: "Chennai", pin: 600001 }
};
user.name                        // dot — the normal way
user["job title"]                // bracket — spaced keys, or DYNAMIC keys: user[keyVar]
user.address.city                // nested access
const { name, address: { city } } = user;  // destructuring
const copy = { ...user, name: "Ravi" };    // spread: copy + override
```

ARRAYS — DECLARE & ACCESS

```
const nums = [10, 20, 30];
nums[0]      // 10 — index starts at 0
nums.length  // 3
nums[nums.length - 1] // 30 — last item
const [first, ...rest] = nums; // destructuring + rest → first=10, rest=[20,30]
const more = [...nums, 40];    // spread: new array, not mutation
```

INTERVIEW Q

"Dot vs bracket access?" — dot for fixed simple keys; bracket when the key has spaces/special characters or lives in a variable.
"for...of vs for...in?" — of iterates values of arrays; in iterates keys of objects.

FUNCTIONS — EVERY FORM

PARAMETERS VS ARGUMENTS · THE THREE FORMS

```
function add(a, b) { return a + b; }    // 1) DECLARATION — hoisted
//      ↑ ↑  PARAMETERS: placeholders in the definition
add(2, 3);
// ↑ ↑  ARGUMENTS: the real values you pass

const add2 = function (a, b) { return a + b; };    // 2) EXPRESSION — not hoisted
const add3 = (a, b) => a + b;                      // 3) ARROW — implicit return
const square = n => n * n;                          //   one param: parens optional
const make = (name) => ({ name });                  //   returning an object: wrap in ()
```

Arrow functions differ in three ways: **no own this** (inherited from where they're written), **no arguments object**, and they **can't be constructors**. Default parameters: `function greet(name = "Guest")`.

CALLBACK FUNCTIONS

A callback is a function **passed as an argument** to be called later — the foundation of array methods, event listeners and async code.

```
function processOrder(item, onDone) {    // onDone is a callback parameter
  console.log("Packing " + item);
  onDone(item);                          // called when work finishes
}
processOrder("Shoes", (it) => console.log(it + " shipped"));
button.addEventListener("click", handleClick);    // callbacks everywhere
setTimeout(() => console.log("later"), 1000);
```

BUILT-IN METHODS & PROTOTYPES

Where does `"hi".toUpperCase()` come from? The **prototype chain**: every string links to `String.prototype`, every array to `Array.prototype` — methods are defined once and shared by all instances.

```
"hello".toUpperCase()    " x ".trim()    "a,b,c".split(",")
(3.14159).toFixed(2)      Math.max(1, 5)    Object.keys(user)    Object.values(user)
```

INTERVIEW Q

"Function declaration vs expression?" — declarations are hoisted (callable before they appear); expressions and arrows are not.
 "What is a callback?" — a function passed as an argument, invoked later; name an example you use daily: `map`, `addEventListener`, `setTimeout`.

ARRAY METHODS, HOF & CLOSURES

THE HIGHER-ORDER ARRAY METHODS

A **higher-order function** takes or returns a function. These are the most-used lines in React codebases:

METHOD	RETURNS	EXAMPLE
map	new array, same length	<code>[1,2,3].map(n => n * 2) → [2,4,6]</code>
filter	new array, matches only	<code>users.filter(u => u.active)</code>
reduce	one value	<code>cart.reduce((sum, i) => sum + i.price, 0)</code>
find	first match / undefined	<code>users.find(u => u.id === 7)</code>
some · every	boolean	<code>nums.some(n => n > 9) · nums.every(n => n > 0)</code>
forEach	undefined — side effects	<code>rows.forEach(r => console.log(r))</code>
sort	same array, mutated!	<code>nums.sort((a, b) => a - b) — numeric ascending</code>
slice · splice	copy · mutation	<code>slice(0,2)</code> copies; <code>splice(1,2)</code> removes in place

INTERVIEW Q

"map vs forEach?" — map returns a new array (chainable; how React renders lists); forEach returns undefined. "Which methods mutate?" — push pop shift unshift splice sort reverse mutate; map filter slice concat return new arrays. React state needs the non-mutating ones.

CLOSURES — THE #1 JS INTERVIEW TOPIC

A closure is a function that **remembers the variables of its outer scope** even after that outer function has finished running. It gives JavaScript private state — and it is exactly how React hooks work under the hood.

```
function counter() {
  let count = 0;           // lives in counter's scope
  return function () {     // this inner fn CLOSES OVER count
    count++;
    return count;
  };
}

const inc = counter();     // counter() has returned...
inc(); inc(); inc();       // ...yet count survives: 1, 2, 3
const inc2 = counter();    // a fresh, independent closure: 1
```

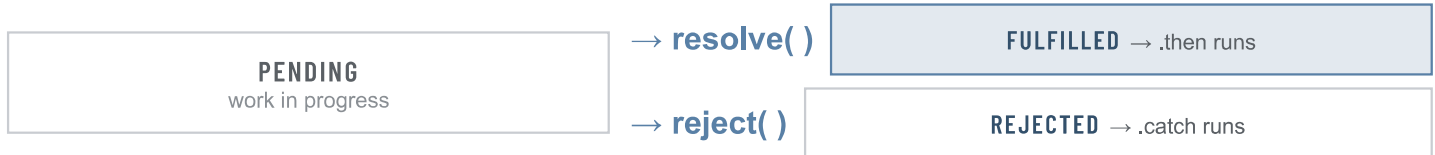
Say in the interview: "count is not global and not accessible from outside — only the returned function can touch it. Debounce, throttle and useState all rely on this."

JSON, PROMISES & CHAINING

JSON — THE WIRE FORMAT

```
const text = '{"name":"Asha","age":24}'; // JSON is a STRING format for data
const obj  = JSON.parse(text);           // string → object   (receiving)
const wire = JSON.stringify(obj);        // object → string   (sending to an API)
```

PROMISE — A FUTURE VALUE WITH THREE STATES



```
const order = new Promise((resolve, reject) => {
  setTimeout(() => {
    stockAvailable ? resolve("packed") : reject(new Error("out of stock"));
  }, 1000);
});
```

PROMISE CHAINING — EACH .THEN RETURNS A NEW PROMISE

```
order
  .then(status => status + " → shipped") // return value flows to next .then
  .then(status => console.log(status))   // "packed → shipped"
  .catch(err => console.error(err.message)) // ONE catch handles the whole chain
  .finally(() => console.log("done either way"));
```

Also know: `Promise.all([p1, p2])` — waits for all, fails fast if any rejects. `Promise.allSettled` — waits for all, never short-circuits.

INTERVIEW Q

"What is a promise?" — "An object representing a value that isn't ready yet. Pending, then fulfilled or rejected. I consume it with `.then/.catch` or `async/await`." Then mention `Promise.all` for parallel requests — that's the senior signal.

FETCH, TRY/CATCH & ASYNC/AWAIT

STYLE 1 — .THEN CHAINING

```
fetch("https://api.shop.com/products")
  .then(res => res.json())           // fetch resolves to a Response object;
  .then(data => renderList(data))    // .json() is ITSELF a promise → second .then
  .catch(err => showError(err));
```

STYLE 2 — ASYNC/AWAIT WITH TRY/CATCH (SAY YOU PREFER THIS)

```
async function loadProducts() {      // async ⇒ always returns a promise
  try {
    const res = await fetch("https://api.shop.com/products");
    if (!res.ok) throw new Error("HTTP " + res.status); // fetch does NOT
    const data = await res.json();                     // reject on 404/500!
    renderList(data);
  } catch (err) {                                     // network failures + thrown errors land here
    showError(err.message);
  } finally {
    hideSpinner();                                   // runs on success AND failure
  }
}
```

await pauses only this function until the promise settles — the main thread keeps running (the event loop handles other work). That's why the UI never freezes during a fetch.

POST WITH FETCH · AXIOS COMPARISON

```
await fetch("/api/orders", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ productId: 7, qty: 2 })
});

// axios: npm i axios
const { data } = await axios.post("/api/orders", { productId: 7, qty: 2 });
```

fetch	Built-in · manual <code>res.json()</code> · does <i>not</i> reject on HTTP 4xx/5xx · manual headers
axios	Library · auto-JSON both ways · rejects on 4xx/5xx · interceptors (attach JWT to every request)

INTERVIEW Q

"async/await vs .then?" — same promises underneath; async/await reads top-to-bottom and uses normal try/catch. "Does fetch reject on a 404?" — No. It only rejects on network failure; check `res.ok` yourself. This trips up most candidates.

EVENTS, DEBOUNCE & SCALE

BUBBLING → EVENT DELEGATION

Events **bubble** upward: click an `<li>` and the event fires on it, then its `<ul>`, then every ancestor. **Delegation** exploits this — one listener on the parent serves all children, including ones added later:

```
list.addEventListener("click", (e) => {
  if (e.target.matches("li")) select(e.target); // one listener, N items
});
// e.stopPropagation() → halt bubbling | e.preventDefault() → block default action
```

DEBOUNCE VS THROTTLE – BOTH ARE CLOSURES

DEBOUNCE – AFTER SILENCE

Search box: wait until typing stops, then fire once.

```
function debounce(fn, delay) {
  let t;
  return (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), delay);
  };
}
```

THROTTLE – STEADY RATE

Scroll handler: fire at most once per interval.

```
function throttle(fn, limit) {
  let ok = true;
  return (...args) => {
    if (!ok) return;
    fn(...args); ok = false;
    setTimeout(() => ok = true, limit);
  };
}
```

SCALE VOCABULARY – ONE LINE EACH

Indexing	A database lookup structure (like a book's index): reads become fast, writes slightly slower.
Load balancing	Spreading incoming requests across several servers so no single one is overwhelmed.
Rate limiting	Capping requests per user per window (100/min) — exceeded requests get HTTP 429.

NIGHT-BEFORE CHECKLIST

=== vs == · falsy six · value vs reference · closures with the counter example · map/filter/reduce · promise states · fetch doesn't reject on 404 · debounce vs throttle · "type a URL" story. If you can say each in one breath, you're ready — continue to Book 02: React.